Project Title: Travel Planning and Policy Assistant
Application Name: TripWise AI
Document Type: Comprehensive Technical & Architectural Report
Version: 1.0.0

Abstract

Planning a multi-day trip requires synthesizing heterogeneous information across destination guides, hotel booking policies, public transit baggage regulations, regional entry advisories, and personal traveller constraints. While Large Language Models (LLMs) excel at fluent text generation, unconstrained conversational travel assistants frequently hallucinate opening hours, fabricate policy citations, overlook contradictory rules across documents, and fail to warn users when source documents are undated or outdated.

1. Introduction & Problem Statement

2. Project Objectives & Core Principles

2.2 Core Non-Fabrication Invariants

TripWise AI enforces five architectural invariants across all backend services and UI views:

- Invariant 1 (No Unbacked Citations): Every CitationReference attached to an itinerary activity or policy check must reference an actual chunk_id present in the current run's retrieved_sources.

- Invariant 2 (Explicit Unverified Labelling): When no curated documents match a requested destination, activities are generated as structural placeholders explicitly labelled "Unverified suggestion ? missing from knowledge base" with estimated_cost = null and citations = [].

- Invariant 3 (No Silent Conflict Suppression): Conflicting chunks across distinct documents are both retained and rendered side-by-side in the Policy Checks and Missing Information panels.

- Invariant 4 (Similarity $\neq$ Certainty): Hybrid retrieval scores are presented strictly as query-document similarity metrics, never as factual confidence percentages.

- Invariant 5 (Safe Failure Semantics): A failed external live lookup or LLM provider timeout never crashes the pipeline and is never interpreted as proof that travel restrictions do not exist.

3. Methodology & Hybrid RAG Pipeline

The TripWise AI knowledge pipeline follows a strict nine-step lifecycle:

$$\text{Upload} \rightarrow \text{Validate} \rightarrow \text{Extract Text} \rightarrow \text{Normalize} \rightarrow \text{Chunk} \rightarrow \text{Embed} \rightarrow \text{Index} \rightarrow \text{Retrieve} \rightarrow \text{Cite}$$

3.1 Document Validation & Security Hardening

Uploaded files are processed by backend/app/retrieval/document_processor.py:

- File Type & Size Guards: Only .pdf, .md, and .txt extensions are accepted, up to a strict 10 MB (10,485,760 bytes) ceiling. Empty (0-byte) payloads are immediately rejected.

- Path Traversal Prevention: Filenames are stripped of directory separators (.., /, \) and sanitized via regular expression whitelisting ([^A-Za-z0-9._\-]).

- Duplicate Detection: A SHA-256 cryptographic hash (content_hash) is computed over raw file bytes; duplicate uploads return HTTP 409 Conflict referencing the existing document_id.

- Prompt-Injection Neutralization: All extracted text is treated as untrusted reference data. Regex filters neutralize adversarial instruction patterns (e.g., ignore previous instructions, system prompt:, execute shell command) into [REDACTED_UNTRUSTED_DIRECTIVE] before chunking or LLM prompting.

3.3 Hybrid Semantic (FAISS) + Lexical (BM25) Retrieval Engine

Travel policy retrieval requires both semantic understanding (matching *"Can I leave my bags at the hotel early in the morning?"* to *"early arrival luggage drop-off before 10:00 AM is prohibited"*) and exact lexical precision (matching *"3:00 PM"*, *"15 kg"*, *"55x40x20 cm"*, or *"INR 650"*).

Implemented in backend/app/retrieval/hybrid_retriever.py, our HybridRetriever combines two indexes:

1. Dense Vector Index (faiss.IndexFlatIP):

Each chunk c_i is formatted with its metadata header ($T_i = \text{title} \mid \text{section} \mid \text{category} \mid \text{destination} \mid \text{content}$) and encoded into an L_2-normalized 384-dimensional dense vector $\mathbf{v}_i \in \mathbb{R}^{384}$ ($\|\mathbf{v}_i\|_2 = 1$). The SemanticEmbedder loads a locally cached SentenceTransformer (all-MiniLM-L6-v2) once per process and provides an instant deterministic 384-dimensional subword n-gram + domain-concept projection fallback so offline execution never blocks on external network downloads. Inner product search in faiss.IndexFlatIP yields exact cosine similarity:

$$S_{\text{sem}}(q, c_i) = \max\left(0, \langle \mathbf{v}_q, \mathbf{v}_i \rangle\right)$$

2. Lexical Okapi BM25 Index with Domain Synonym Expansion:

Query tokens q are expanded with curated travel domain synonyms (e.g., hotel $\rightarrow$ {accommodation, check-in, check-out, room, inn}; transport $\rightarrow$ {metro, bus, transit, luggage, baggage, pass}) with a 0.45 expansion weight multiplier. For corpus size N and average chunk length $\bar{L}$, the unnormalized BM25 score is:

$$\text{BM25}(q, c_i) = \sum_{t \in q} w_t \cdot \ln\left(1 + \frac{N - df(t) + 0.5}{df(t) + 0.5}\right) \cdot \frac{tf(t, c_i) \cdot (k_1 + 1)}{tf(t, c_i) + k_1 \cdot \left(1 - b + b \cdot \frac{|c_i|}{\bar{L}}\right)}$$

where $k_1 = 1.5$ and $b = 0.75$. Scores are max-normalized across the candidate set:

$$S_{\text{kw}}(q, c_i) = \frac{\text{BM25}(q, c_i)}{\max_{j} \text{BM25}(q, c_j) + \epsilon}$$

3. Hybrid Fusion & Deduplication:

The final hybrid relevance score is computed as a convex combination:

$$S_{\text{hybrid}}(q, c_i) = 0.55 \cdot S_{\text{sem}}(q, c_i) + 0.45 \cdot S_{\text{kw}}(q, c_i)$$

Candidate chunks are filtered by destination and category, deduplicated by both chunk_id and normalized content MD5 hash, and assigned an epistemic verification_status label based on document metadata:

- publication_date is None $\rightarrow$ "Document date unknown"
- publication_date < "2025-01-01" $\rightarrow$ "Requires official confirmation"
- Otherwise $\rightarrow$ "Verified in provided documents"

4. Multi-Stage State-Graph Workflow Architecture (LangGraph-Style Pipeline)

Rather than executing a single monolithic prompt, TripWise AI models trip planning as a directed, six-node State-Graph Workflow (TripPlanningWorkflow in backend/app/workflow/pipeline.py). A strongly-typed state container progresses through nodes 1 to 6, recording per-node execution status (pending, running, completed, completed_with_warnings, failed), ISO-8601 start/completion timestamps, warnings, and structured JSON outputs.

mermaid

```
flowchart TD
    Input(["User Input: TravellerPreferences"]) --> N1
```

```
subgraph StateGraph["6-Node State-Graph Execution Pipeline (TripPlanningWorkflow)"]
    N1["Node 1: Preference Identification\n(NormalizedTravelProfile)"]
    N2["Node 2: Curated Knowledge Retrieval\n(Multi-Query FAISS + BM25)"]
    N3["Node 3: Itinerary & Budget Generation\n(ItineraryDays + Citations + BudgetBreakdown)"]
    N4["Node 4: Policy & Restriction Validation\n(PASS / WARNING / FAIL / UNKNOWN)"]
    N5["Node 5: Missing-Info & Freshness Audit\n(Undated, Outdated, Conflicts & Live Adapter)"]
    N6["Node 6: Final Plan Assembly\n(FinalTripPlan + Explainability Audit)"]


N1 --> N2 --> N3 --> N4 --> N5 --> N6
N6 --> end Output(["Persisted FinalTripPlan + WorkflowRun in SQLite"])
```

4.1 Node-by-Node Specification

1. Node 1 ? Preference Identification (stage_1_identify_preferences):

- Validates date format (YYYY-MM-DD) and enforces $\text{return_date} \ge \text{departure_date}$ (raising HTTP 422 otherwise).

- Computes inclusive trip duration $D = (\text{return_date} - \text{departure_date}).\text{days} + 1$ and daily per-person budget $B_{\text{daily}} = \frac{\text{budget_total}}{D \times \text{travellers_count}}$.

- Detects internal preference contradictions:

- *Traveller Count Mismatch:* $\text{adults} + \text{children} \neq \text{travellers_count}$.

- *Style?Budget Contradiction:* Selecting luxury travel style with budget_hotel or hostel or $B_{\text{daily}} < \text{INR } 2,500$.

- *Pace?Interest Tension:* Selecting relaxed pace alongside ≥ 6 interest categories.

2. Node 2 ? Curated Knowledge Retrieval (stage_2_retrieve_knowledge):

- Dispatches 5?6 targeted hybrid queries covering (i) sightseeing highlights & durations, (ii) hotel check-in/out & cancellation rules, (iii) transit passes & baggage limits, (iv) entry permits & identification, (v) accessibility & safety rules, plus any user-supplied constraint text.

- Merges and deduplicates chunks by retaining the highest retrieval_score per chunk_id.

3. Node 3 ? Destination Research & Itinerary Generation (stage_3_generate_itinerary):

- Constructs ItineraryDay schedules with Morning, Afternoon, and Evening activities.

- Binds retrieved CitationReference objects to each activity via keyword-category alignment.

- Computes BudgetBreakdown:

$$\text{Cost}_{\text{total}} = \left(\text{Lodging} + \text{TransitPass} + \text{Entries} + \text{Dining}\right) \times 1.08_{\text{buffer}}$$

- Uses conservative budgeting when conflicting tariffs exist (e.g., applying the revised INR 650 museum admission fee rather than the older INR 500 figure).

4. Node 4 ? Policy & Restriction Validation (stage_4_validate_policies):

- Evaluates up to 8 distinct policy rules across retrieved evidence:

- *Budget Compliance:* PASS if $\text{Cost}_{\text{total}} \le \text{budget_total}$ with retrieved tariff support; FAIL if exceeded; UNKNOWN if no pricing docs exist.

- *Early Arrival & Check-In Compliance:* Inspects traveller notes for early arrival requests (8:30 AM, before 10, early check-in, luggage drop) against retrieved accommodation rules (check-in 3:00 PM, luggage drop-off before 10:00 AM prohibited). Produces a documented FAIL when conflicting, or PASS when arrival aligns with standard hours.

- *Adult Age (18+) & Photo ID Compliance:* Produces PASS when supported by retrieved hotel policy.

- *Public Transit Peak-Hour Baggage Rule:* Produces WARNING citing Page 2 of the transport policy (1 cabin bag <= 55x40x20 cm, 15 kg during 8:00?10:30 AM and 5:30?8:00 PM).

- *Cross-Document Tariff & Hours Conflict:* Compares chunks across documents; when sample-destination-001 (INR 500, 5:00 PM) and sample-annex-006 (INR 650, Friday 4:00 PM) are both retrieved, issues a WARNING populating both supporting_document_title and conflicting_document_title.

- *Entry Permit & 24-Hour Digital Eco-Pass:* Issues WARNING when the entry guidance document is dated prior to 2025 (2024-06-01).

- *Live Transit Headways & Real-Time Status:* Issues UNKNOWN because static manuals exclude live departure timetables.

5. Node 5 ? Missing-Information & Freshness Checker (stage_5_check_missing_and_freshness):

- Audits all retrieved chunks and policy checks to generate MissingInformation items classified by severity (critical, high, medium, low) and status label (Verified in provided

5. System Architecture & Technology Stack

5.1 Technology Stack Summary

Component	Technology	Role in TripWise AI
Frontend Framework	React 18.3 + TypeScript 5.6 + Vite 6	Component-driven UI, strict static typing, fast HMR
Styling & Icons	Tailwind CSS 3.4 + Lucide React	Travel-editorial aesthetic, responsive layout, print/PDF styles
Visual Analytics	Recharts 2.15	Interactive budget breakdown bar charts & financial utilization
Backend Framework	Python 3.14 + FastAPI + Pydantic v2	REST API endpoints, schema validation, OpenAPI/Swagger docs
Document Parsing	PyMuPDF (pymupdf 2.0) + Custom Parsers	Page-level PDF text extraction, Markdown/TXT section chunking
Vector & Lexical Search	faiss-cpu (IndexFlatIP) + sentence-transformers + BM25	Local hybrid semantic + keyword retrieval
Database	SQLite 3 (tripwise.db)	Parameterized relational storage for trips, documents, chunks, runs
Automated Testing	pytest 9.2 + Vitest 2.1 + React Testing Library	18-point backend test suite + frontend component/validation tests

5.2 Relational Database Schema (backend/app/db/database.py)

The SQLite persistence layer manages four tables with PRAGMA foreign_keys = ON:

1. documents: Stores document_id (PK), title, filename, file_type, file_size_bytes, category, authority, publication_date, last_verified_date, destination, is_demonstration, processing_status, chunk_count, page_count, content_hash (indexed for duplicate detection), created_at, summary_excerpt, and full_text.

2. document_chunks: Stores chunk_id (PK), document_id (FK with ON DELETE CASCADE), document_title, category, authority, destination, page_number, section_heading, chunk_index, content, publication_date, last_verified_date, and is_demonstration.

3. trips: Stores trip_id (PK), title, destination, created_at, updated_at, verification_summary_status, is_fully_verified, and plan_json (serialized FinalTripPlan).

4. workflow_runs: Stores run_id (PK), trip_id (FK with ON DELETE CASCADE), overall_status, started_at, completed_at, and run_json.

6. Evaluation Strategy, Experimental Setup & Observed Results

6.1 Seeded Curated Corpus (*Sample City* Scenario)

To ensure deterministic, reproducible faculty evaluation without fabricating real-world government rules, the system seeds six clearly labelled demonstration documents in backend/data/sample_documents/:

Document ID	Title & Format	Category	Pages / Chunks	Publication Date	Designed Evaluation Purpose
sample-destination-001	*Sample City Cultural & Sightseeing Handbook* (.md)	destination_guide	2 pages / 9 chunks	2026-01-15	Sightseeing durations, meal costs, North Watchtower seasonal closure, INR 500 museum tariff
sample-accommodation-001	*Harbourview Budget Inn Policy* (.pdf)	accommodation_policy	2 pages / 5 chunks	null (Undated)	Real 2-page PDF (INR 3,200/night, Page 2: check-in 3:00 PM, < 10:00 AM luggage ban, undated warning)
sample-transport-001	*Sample City Metro & Transit Rider Rules* (.md)	transport_policy	2 pages / 5 chunks	2025-11-10	INR 450 3-day pass, peak-hour 15 kg cabin bag limit, missing live train headways
sample-entry-001	*Regional Entry, Permit & Advisory Guidelines* (.txt)	visa_and_entry_guidance	1 page / 3 chunks	2024-06-01 (Outdated)	Domestic photo ID rule, mandatory 24h Old Quarter Digital Eco-Pass, pre-2025 freshness warning
sample-safety-001	*Accessibility, Health & Safety Handbook* (.md)	accessibility_guidance	1 page / 3 chunks	2026-01-20	Step-free elevators vs. Old Quarter cobblestones, Green Hygiene Seal food stalls, drone ban
sample-annex-006	*Municipal Heritage Circular ? Revised Tariff* (.txt)	safety_and_travel_restrictions	1 page / 2 chunks	2026-02-10	Revises National Museum entry to INR 650 and Friday closing to 4:00 PM (conflict test)

6.3 Automated Test Suite Matrix (18 Mandatory Verification Requirements)

All 18 testing requirements specified in the project rubric are verified in backend/tests/test_tripwise.py (13 passed) and frontend/src/__tests__/App.test.tsx (4 passed):

Req #	Test Case Description	Test Function	Observed Result
1	Preference normalization & contradiction detection	test_01_preference_validation_and_contradictions	PASSED
2	Invalid date ranges (return_date < departure_date)	test_02_invalid_date_ranges & App.test.tsx	PASSED (HTTP 422)
3	Budget constraint evaluation (PASS vs. FAIL)	test_03_budget_constraint_evaluation	PASSED
4	Document upload validation (size, extensions, path traversal)	test_04_document_upload_validation	PASSED (HTTP 400)
5	PDF (PyMuPDF) & text extraction + scanned PDF rejection	test_05_pdf_and_text_extraction	PASSED
6	Document chunking & page/section metadata preservation	test_06_chunking_and_metadata_preservation	PASSED
7	Hybrid semantic (FAISS) + keyword (BM25) retrieval	test_07_semantic_and_keyword_retrieval	PASSED
8	Citation accuracy & zero fabricated citations on missing docs	test_08_citations_and_no_fabrication_on_missing_docs	PASSED
9	Policy PASS, WARNING, FAIL, and UNKNOWN outcomes	test_09_policy_pass_warning_fail_unknown_outcomes	PASSED
10	Missing-information detection	test_10_11_12_missing_conflicting_outdated_and_undated_detection	PASSED
11	Multi-document conflicting source detection	test_10_11_12_missing_conflicting_outdated_and_undated_detection	PASSED
12	Outdated (2024) & undated (null) document warnings	test_10_11_12_missing_conflicting_outdated_and_undated_detection	PASSED
13	Missing AI API credentials (local deterministic mode)	test_13_14_missing_credentials_and_ai_provider_failure	PASSED
14	External AI provider network failure / timeout fallback	test_13_14_missing_credentials_and_ai_provider_failure	PASSED
15	Workflow stage failure handling & SSRF/live lookup safety	test_15_workflow_stage_failure_and_live_adapter_safety	PASSED
16	Saving, updating, and replanning trips via REST API	test_16_17_18_end_to_end_api_trips_and_document_lifecycle	PASSED
17	Document deletion & SQLite/FAISS chunk cleanup	test_16_17_18_end_to_end_api_trips_and_document_lifecycle	PASSED
18	End-to-end trip planning on seeded demonstration scenario	test_16_17_18_end_to_end_api_trips_and_document_lifecycle	PASSED

7. System Limitations & Future Enhancements

7.2 Future Enhancements

1. Vision-LLM & Tesseract OCR Integration: Automatically fall back to OCR page extraction when PyMuPDF detects image-only PDF pages.

2. Graph-Based Multi-City Routing: Extend the 6-stage state graph with a cyclic re-routing node that automatically shifts museum visits across days if a weekday closure conflict is detected (e.g., Sample City National Museum closed on Mondays).

3. Automated Currency Conversion Adapter: Integrate a cached exchange-rate feed to normalize multi-currency documents against the traveller's preferred budget currency.

8. References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459?9474.
2. Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333?389.
3. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-Scale Similarity Search with GPUs (FAISS)*. IEEE Transactions on Big Data, 7(3), 535?547.
4. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
5. LangChain AI (2024). *LangGraph: Building Language Agents as Graphs*. Technical Documentation on State-Graph Agentic Workflows.
6. Ramírez, S. (2024). *FastAPI: Modern, Fast Web Framework for Building APIs with Python based on Standard Type Hints*. https://fastapi.tiangolo.com/
7. PyMuPDF Developers (2025). *PyMuPDF Documentation: High-Performance Python Bindings for MuPDF*. https://pymupdf.readthedocs.io/